

Wanderly — AI-Powered Travel Agency Platform

Complete Project Plan

Stack: FastAPI · Next.js · PostgreSQL · Redis · LLM Agent · Duffel Flights API

1. Project Goals

The project has two jobs.

For users, it should make trip planning effortless: describe a trip in plain language, get real flight options and a day-by-day itinerary, then book.

For the Upwork portfolio, it should prove the ability to build production-quality FastAPI backends, integrate LLMs with tool calling and streaming, work with third-party APIs, and deliver a polished UI that non-technical clients love.

Scope rule: build an MVP that works end to end first, then add extras. A finished, smooth, smaller app beats an unfinished big one every time.

2. Tech Stack

Layer	Choice	Why
Frontend	Next.js (App Router) + TypeScript	Modern, fast, SEO-friendly, most requested by clients
UI	Tailwind CSS + shadcn/ui + Framer Motion + Lucide icons	Premium look with little effort, smooth animations
Data fetching	TanStack Query	Caching, loading states, retries built in
Forms	React Hook Form + Zod	Clean validation on search and booking forms
Maps	Leaflet + OpenStreetMap	Free, no billing setup
Backend	FastAPI (async) + Python 3.12	Core skill being showcased
Database	PostgreSQL + SQLAlchemy 2.0 (async) + Alembic	Industry standard, migrations
Cache & jobs	Redis + ARQ	Search caching, price-alert worker
Validation	Pydantic v2	Request/response models and LLM output validation
LLM	Claude or OpenAI, behind a custom abstraction layer	Tool calling and streaming; swappable provider
Flights	Duffel API (test mode)	Full search-to-booking in a sandbox
Weather	Open-Meteo	Free, no API key
Images	Unsplash API	Beautiful destination photos
Airports	OurAirports dataset (seeded into Postgres)	Fast, free autocomplete
Email	Resend	Simple transactional email for alerts and confirmations
Auth	JWT (access + refresh tokens), bcrypt/argon2	Shows auth built from scratch
DevOps	Docker Compose, GitHub Actions, pytest	Professional signal in the repo
Hosting	Vercel (frontend), Railway or Render (API, Postgres, Redis)	Cheap or free, easy deploy

Note on flight data: Amadeus decommissioned its Self-Service developer portal on July 17, 2026 (only Enterprise contracts remain), so older tutorials based on it no longer work. Duffel test mode, with its sandbox airline “Duffel Airways” (IATA code ZZ), supports a reliable search-to-booking demo.

3. Features

MVP (must have)

- 1. **Landing page** with hero image, search box, popular destinations, and a “Plan with AI” call to action.
- 2. **Flight search**: one-way and round trip, airport autocomplete, date picker, passengers, cabin class.
- 3. **Results page**: flight cards, filters (stops, airlines, price, times), sorting (cheapest, fastest, best), and an AI summary of the options at the top.
- 4. **Flight details and booking flow**: passenger details form, review, confirmation (Duffel test mode, fake payment).
- 5. **AI trip planner chat**: streaming responses, tool calling for flight search, weather, and itinerary creation, with flight cards rendered inside the chat.
- 6. **Itinerary view**: day-by-day timeline with a map, saved to the user’s account.
- 7. **User accounts**: sign up, log in, “My Trips” (bookings and saved itineraries).

Phase 2 (strong extras)

- 8. **Price alerts** with a daily background check and email notification.
- 9. **Admin dashboard**: bookings, users, popular routes, AI token usage and cost.
- 10. **Share itinerary** via public link and export to PDF.
- 11. **Dark mode** and full mobile polish.

Phase 3 (nice to have)

- 12. Hotels through Duffel Stays.
- 13. Multi-language support (e.g. English plus Urdu or Arabic for Middle East and South Asia clients).
- 14. Voice input in the AI chat.

4. Pages and User Flows

Sitemap

/	Landing page
/search	Flight results (query params: from, to, dates, pax)
/flights/[offerId]	Flight details
/booking/[offerId]	Passenger details -> review -> confirm
/booking/success/[id]	Confirmation
/plan	AI trip planner (chat + itinerary side panel)
/trips	My Trips (bookings, itineraries, alerts)
/trips/[id]	Itinerary detail (timeline + map)
/share/[slug]	Public shared itinerary
/login, /register	Auth
/admin	Admin dashboard (role-protected)

Key Flow 1: Classic search and book

Landing → enter route and dates → results with skeleton loaders → filter and sort → pick a flight → enter passenger details → review price breakdown → confirm → success page with booking reference → booking appears in My Trips.

Key Flow 2: AI planning (the “wow” demo)

User opens /plan → types “5 days somewhere warm in December from Lahore, budget \$800” → AI asks one clarifying question if needed → suggests 2-3 destinations with photos and weather → user picks one → AI searches flights and shows cards in the chat → AI generates the itinerary, which appears in the side panel with a map → user refines (“more food spots on day 2”) → saves the trip or clicks “Book this flight,” which jumps into the normal booking flow.

5. System Architecture



Key design principle — layering:

- **Routers** handle HTTP only.
- **Services** hold business logic.
- **Clients** wrap external APIs.
- **Repositories** talk to the database.

This keeps the code clean and testable, which is exactly what technical clients look for in a repo.

6. Backend Project Structure

```
backend/
├── app/
│   ├── main.py                # App factory, middleware, router registration
│   └── core/
│       ├── config.py          # Pydantic Settings (env vars)
│       ├── security.py         # JWT, password hashing
│       ├── logging.py          # Structured JSON logging
│       └── exceptions.py       # Custom errors + global handlers
├── db/
│   ├── session.py             # Async engine and session
│   └── base.py
├── models/                    # SQLAlchemy models
│   ├── user.py
│   ├── booking.py
│   ├── trip.py
│   ├── conversation.py
│   ├── price_alert.py
│   └── airport.py
├── schemas/                   # Pydantic request/response models
├── repositories/              # DB queries
├── clients/                   # External API wrappers
│   ├── duffel.py
│   ├── weather.py
│   ├── unsplash.py
│   └── email.py
├── services/                  # Business logic
│   ├── flight_service.py
│   ├── booking_service.py
│   ├── trip_service.py
│   └── alert_service.py
├── ai/
│   ├── llm_client.py          # Provider abstraction (Claude / OpenAI)
│   ├── agent.py               # Agent loop: LLM <-> tools
│   ├── tools.py               # Tool definitions + handlers
│   ├── prompts/               # Versioned prompt templates
│   └── schemas.py             # Validated LLM outputs (itinerary, etc.)
├── api/
│   ├── deps.py                # get_db, get_current_user, admin guard
│   └── v1/
│       ├── auth.py
│       ├── airports.py
│       ├── flights.py
│       ├── bookings.py
│       ├── chat.py
│       ├── trips.py
│       ├── alerts.py
│       └── admin.py
├── workers/
│   └── tasks.py                # ARQ jobs: check prices, send emails
├── alembic/
├── scripts/seed_airports.py
├── tests/
│   ├── unit/
│   └── integration/
├── Dockerfile
├── pyproject.toml
└── .env.example
```

7. Database Schema

Table	Key columns
users	id (UUID), email (unique), password_hash, full_name, role (user/admin), created_at
airports	iata_code (PK), name, city, country, country_code, lat, lng, is_major
bookings	id, user_id → users, duffel_order_id, booking_reference, status, total_amount, currency, origin, destination, departure_at, return_at, passengers (JSONB), raw_offer (JSONB), created_at
trips	id, user_id → users, title, destination, start_date, end_date, budget, itinerary (JSONB), share_slug (unique, nullable), is_public, created_at, updated_at
conversations	id, user_id → users (nullable for guests), trip_id → trips (nullable), title, created_at
messages	id, conversation_id → conversations, role (user/assistant/tool), content, tool_name, tool_payload (JSONB), created_at
price_alerts	id, user_id → users, origin, destination, departure_date, return_date, target_price, last_price, currency, is_active, last_checked_at
llm_usage	id, user_id, conversation_id, model, input_tokens, output_tokens, cost_usd, latency_ms, created_at

Storing the itinerary as JSONB keeps it flexible. The `llm_usage` table powers the cost section of the admin dashboard — a detail that impresses clients who worry about AI bills.

8. API Endpoints (v1)

Method	Endpoint	Purpose
POST	/auth/register	Create account
POST	/auth/login	Get access + refresh tokens
POST	/auth/refresh	Refresh access token
GET	/auth/me	Current user
GET	/airports/search?q=	Autocomplete (city, name, or IATA code)
POST	/flights/search	Search offers (cached in Redis ~10 min)
GET	/flights/offers/{id}	Offer details (price rechecked)
POST	/flights/summary	AI summary of a result set
POST	/bookings	Create booking via Duffel
GET	/bookings	User's bookings
GET	/bookings/{id}	Booking details
POST	/chat/conversations	Start conversation
POST	/chat/conversations/{id}/messages	Send message, streamed via SSE
GET	/chat/conversations/{id}	Conversation history
GET / POST	/trips	List or create trips
GET / PATCH / DELETE	/trips/{id}	Manage a trip
POST	/trips/{id}/share	Generate public link
GET	/public/trips/{slug}	View shared itinerary
GET / POST / DELETE	/alerts	Manage price alerts
GET	/admin/stats	Bookings, users, top routes, AI costs
GET	/health	Health check for deployment

FastAPI auto-generates Swagger docs at `/docs`. Keep them public on the demo — technical clients like clicking through them.

9. AI Design (the Heart of the Project)

Agent Loop

1. The user message and conversation history are sent to the LLM along with tool definitions.
2. If the LLM calls a tool, the backend runs it and sends the result back.
3. This repeats until the LLM produces a final answer.
4. Everything streams to the frontend over SSE, including progress events like “Searching flights...” so the user sees what is happening.

Tools the Agent Can Use

Tool	What it does
<code>search_airports</code>	Resolve “Lahore” or “somewhere in Bali” to IATA codes
<code>search_flights</code>	Call Duffel, return a compact list of the top offers
<code>get_weather</code>	Forecast or seasonal averages from Open-Meteo
<code>suggest_destinations</code>	Structured destination ideas based on budget, season, interests
<code>create_itinerary</code>	Generate a day-by-day plan as validated JSON
<code>update_itinerary</code>	Apply a change (“make day 3 relaxed”) to the saved plan

Streaming Event Format

Clear event types let the frontend render rich UI, not just text:

<code>event: text</code>	-> token chunk for the chat bubble
<code>event: tool_start</code>	-> "Searching flights from LHE to DXB..."
<code>event: flight_cards</code>	-> JSON list, rendered as cards in chat
<code>event: itinerary</code>	-> JSON, updates the side panel + map
<code>event: done</code>	-> final message id, token usage
<code>event: error</code>	-> friendly error message

Reliability and Guardrails

These separate a production-grade project from hobby demos, so mention them in the portfolio write-up:

- Validate every structured LLM output with Pydantic; retry once with the validation error if it fails.
- Trim flight results before sending them to the LLM (top 5 offers, key fields only) to cut tokens and cost.
- Cap tool-call rounds per message (e.g. 5) to prevent loops.
- Add timeouts and retries (tenacity) on all external calls.
- Keep the system prompt versioned in `prompts/`, and instruct the model never to invent prices or flight times — only use tool results.
- Rate-limit the chat endpoint per user or IP, since the demo is public and LLM calls cost money.
- Log tokens, cost, and latency per request into `llm_usage`.

LLM Provider Abstraction

Write a small `LLMClient` interface with `stream_chat(messages, tools)` and implement it for one provider first. “Provider-agnostic — switch between Claude and OpenAI with one environment variable” is a strong selling point.

10. Frontend Structure

```
frontend/
├── app/
│   ├── (marketing)/page.tsx      # Landing
│   ├── search/page.tsx
│   ├── flights/[offerId]/page.tsx
│   ├── booking/[offerId]/page.tsx
│   ├── booking/success/[id]/page.tsx
│   ├── plan/page.tsx            # AI planner
│   ├── trips/page.tsx
│   ├── trips/[id]/page.tsx
│   ├── share/[slug]/page.tsx
│   ├── admin/page.tsx
│   └── (auth)/login, register
├── components/
│   ├── ui/                     # shadcn components
│   ├── search/                 # SearchBar, AirportCombobox, DateRangePicker, PassengerPicker
│   ├── flights/               # FlightCard, FilterSidebar, SortTabs, AISummaryBanner
│   ├── booking/               # PassengerForm, PriceBreakdown, Stepper
│   ├── chat/                  # ChatWindow, MessageBubble, ToolStatus, ChatFlightCard
│   ├── itinerary/             # Timeline, DayCard, TripMap
│   └── layout/                 # NavBar, Footer, ThemeToggle
├── lib/
│   ├── api.ts                 # Typed API client
│   ├── sse.ts                 # SSE stream parser for chat
│   └── utils.ts
└── hooks/                     # useFlightSearch, useChatStream, useAuth
```

Design System

- One primary brand color (a confident teal or deep blue suits travel) plus one warm accent for prices and calls to action, used consistently.
- A clean font such as Inter or Plus Jakarta Sans.
- Generous spacing, rounded corners (12–16px), soft shadows.
- Prices large and bold — price is what users scan for first.

UX Details That Impress Non-Technical Clients

- Skeleton loaders shaped like flight cards while searching, with rotating messages like “Checking 300+ airlines...”
- Airport autocomplete showing city, airport name, and country flag.
- A sticky mini search bar on the results page so users can tweak without going back.
- Empty and error states with friendly illustrations instead of plain text.
- Subtle page transitions with Framer Motion.
- On mobile: a bottom sheet for filters and a full-screen chat.
- Toast notifications for actions like “Trip saved” or “Alert created.”

11. Development Roadmap (about 8 weeks part-time)

Week	Focus	Deliverable
1	Setup and foundation	Repos, Docker Compose (API, Postgres, Redis), FastAPI skeleton, config, logging, Alembic, Next.js + Tailwind + shadcn, design system basics
2	Auth and airports	Register/login/JWT, seed airports, autocomplete endpoint and UI, landing page and hero
3	Flight search	Duffel client, search endpoint with Redis caching, results page with cards, filters, sorting, skeletons
4	Booking flow	Offer details, passenger form, create order in test mode, success page, My Trips bookings list
5	AI chat core	LLM abstraction, agent loop, tools (airports, flights, weather), SSE streaming, chat UI with tool status and flight cards
6	Itineraries	Itinerary tool with Pydantic validation, side panel timeline and map, save, edit via chat, share links
7	Alerts and admin	ARQ worker, daily price checks, Resend emails, admin dashboard with stats and AI cost charts, AI results summary
8	Polish and launch	Mobile polish, dark mode, tests, GitHub Actions CI, deployment, seed demo data, README, demo video

If time is tight, weeks 1–6 alone make a strong portfolio piece; alerts and admin can be added later as an update.

12. Testing, Quality, and Deployment

Testing

- pytest + pytest-asyncio for services and endpoints.
- Mock external APIs (Duffel, LLM) with respx so tests are fast and free.
- A handful of integration tests for the full booking flow against Duffel test mode.
- A few Playwright tests on the frontend for search and booking.

Code quality

- Ruff for linting and formatting, mypy for type checking, pre-commit hooks.
- ESLint + Prettier on the frontend.
- GitHub Actions runs lint and tests on every push; a green CI badge in the README looks professional.

Deployment

- Frontend on Vercel; API and worker on Railway or Render with managed Postgres and Redis.
- Secrets in environment variables with a documented `.env.example`.
- CORS restricted to the frontend domain; `/health` endpoint for uptime checks.

Demo safety

- “Try demo account” button for one-click login.
- Rate-limit the AI chat and set a daily spending cap in the LLM provider console.
- A small banner stating bookings are in test mode.

13. Estimated Running Costs

Item	Approximate cost
Duffel test mode, Open-Meteo, OurAirports, Leaflet	Free
Vercel (hobby tier)	Free
Railway / Render with Postgres and Redis	~\$5–20 / month
LLM usage (mid-tier model, rate-limited demo)	A few dollars / month
Resend, Unsplash	Free tiers cover a demo

Verify each provider’s current pricing before starting, as these change.

14. Presenting It on Upwork

Portfolio title: “AI-Powered Travel Booking Platform (FastAPI + Next.js + LLM Agent)”

Media

- Cover image showing the AI chat and itinerary side by side.
- 3–4 clean screenshots: landing, results, chat, admin dashboard.
- A 2–3 minute Loom video that opens with the AI planning a trip in the first 30 seconds.

Description (framed around business value)

- Customers can plan and book trips 24/7 by chatting with an AI assistant.
- The agency gets a dashboard showing bookings and AI costs.
- The system searches real airline inventory through a flight API.
- It is built to be reliable: validated AI outputs, caching, background jobs, and tests.

Links

- Live demo with the one-click demo account.
- GitHub repo with an architecture diagram and setup instructions.
- Public Swagger API docs.

Next step: Week 1 — generate the FastAPI project skeleton with Docker Compose, configuration, database setup, and initial migrations.